Deep Reinforcement Learning for Energy-Efficient Cloud Resource Management

Group 2 — An Extension of DDQN-TS (Tong et al., Neurocomputing 455, 2021)

1st Aya Mohamed Mahmoud *Queen's University* 25jhlq@queensu.ca

2nd Fatma Ahmed Mansour *Queen's University* 25nyln@queensu.ca

3rd Manar Fathy Elshenawy *Queen's University* 25ssyn@queensu.ca

4th Natalie Monged Maurice *Queen's University* 25@queensu.ca

***Abstract*—This paper presents a comprehensive paper-expansion study of the DDQN-TS cloud scheduling algorithm by Tong et al. [1]. We first replicate the original framework, correcting three implementation bugs (idle-time tracking, dimensional state inconsistencies, and reward normalization scaling), and benchmark DDQN-TS against vanilla DQN, Round Robin, and Random schedulers across three real-world trace datasets (Random, Google, Alibaba) at task volumes from $n = 1,000$ to $n = 8,000$. The replication reveals a critical training instability artifact: both custom-rolled RL agents collapse under high task loads and exhibit identical policy degeneracy in the 8-VM stress-test, consistent with gradient explosions within the paper's vanilla SGD optimizer without gradient clipping. We then extend the baseline by constructing a new, independently-validated, multi-objective, energy-aware Gymnasium simulator whose $5N+6$ state space integrates CPU/RAM/Power telemetry. The reward function jointly optimizes throughput T, latency L, and energy cost E via a normalized scalarization amplified by an empirical scale factor ($c = 10$). Within this environment, we conduct an algorithmic expansion evaluating Stable-Baselines3 DQN against Proximal Policy Optimization (PPO). Evaluated across 20 independent episodes under out-of-distribution Poisson workload intensities ($\lambda = 0$–3) after training at $\lambda = 1.0$, DQN achieves a peak mean reward of 207.5 at $\lambda = 2.0$ while untuned PPO tops out at 171.0, demonstrating that default DQN converges more rapidly than PPO under constrained sample budgets (200k timesteps). Finally, a reward-weight sensitivity sweep across 10 evaluation episodes shows that the throughput-priority setting ($w_1 = 0.70$) yields the highest empirical rewards for both algorithms, while the energy-priority setting ($w_3 = 0.60$) incurs the largest latency penalty ($\approx 11,880$ ms), mapping a clear multi-objective performance-energy trade-off curve.**

***Index Terms*—Cloud Workload Scheduling, Deep Reinforcement Learning (DRL), Double Deep Q-Networks (DDQN), Proximal Policy Optimization (PPO), Energy-Efficient Green Computing, Multi-Objective Optimization, Gymnasium Simulator, Performance-Energy Trade-off, Operational Scaling Instability, Out-of-Distribution (OOD) Generalization.**

I. Introduction

Efficient resource management in cloud data centres is a sequential decision-making problem under uncertainty. Workload arrival patterns are stochastic, server capacities are heterogeneous, and optimisation targets—task throughput, latency, and power consumption—are jointly conflicting. Traditional heuristic schedulers such as Round Robin (RR) or Greedy Min-Min assign tasks deterministically, without learning from environmental feedback, and are conventionally assumed to be brittle under dynamic load fluctuations—a standard literature assumption that our replication results substantially complicate.

Recent work has proposed deep reinforcement learning (DRL) as an adaptive scheduling framework. The DDQN-TS algorithm by Tong et al. [1] demonstrated that a Double Deep Q-Network (DDQN) can outperform vanilla DQN and several heuristics on a bi-objective scheduling problem minimising makespan and resource imbalance. However, the original paper's scope was narrow: the state space captured only VM idle times, the reward considered no energy metrics, and experimental robustness was limited to a single workload configuration.

This work follows a structured two-phase Paper Expansion approach. In **Phase 0** (Exact Replication), we faithfully replicate the original DDQN-TS framework using its low-dimensional discrete-event layout. This phase corrects three bugs discovered during implementation and documents three unexpected empirical findings:

(a) training instability of vanilla SGD in higher-dimensional task states,
(b) heuristic dominance (specifically Round Robin) over RL agents as task volume scales, and
(c) policy degeneracy in the 8-VM stress test—where findings (a) and (c) represent the mechanism and direct consequence of an underlying gradient stabilization failure.

Phase 1 (Algorithmic Extension) addresses these limitations by introducing a newly constructed, modular, energy-aware Gymnasium simulator with an augmented $5N + 6$ dimensional observation space integrating CPU, RAM, and power telemetry. Within this environment, we conduct an algorithmic expansion evaluating Stable-Baselines3 DQN against Proximal Policy Optimization (PPO) under out-of-distribution Poisson workload stress, providing a systematic reward-weight sensitivity analysis that maps a clear multi-objective performance-energy trade-off curve.

II. Related Work

The scheduling of tasks in virtualised cloud environments has been extensively studied. Classical heuristics such as Min-Min, Max-Min, and Round Robin remain competitive

benchmarks owing to their predictability, execution speed, and zero training cost [2]. The adoption of reinforcement learning for cloud scheduling was pioneered by Mao et al. [3], who demonstrated that a policy-gradient agent could significantly reduce job completion times in cluster environments by learning to dynamically match multi-resource demands to available infrastructure slots.

Tong et al. [1] proposed DDQN-TS, encoding the cloud scheduling problem as an MDP where the state is a low-dimensional vector of VM idle times and the action selects a destination VM for each arriving task. To reduce Q-value overestimation, the algorithm applies the Double DQN correction, alongside a theoretically proposed Thompson Sampling-inspired exploration component. However, for replication tractability and to strictly isolate the mathematical effect of the value-based correction from exploration-strategy variance, our replication framework standardizes exploration across all value-based agents (DDQN-TS and DQN) into an identical, controlled ϵ-greedy strategy. While their experimental results against DQN on three real-world traces demonstrated initial gains in Task Completion Rate (TCR) and Average Response Time (ART), their layout omitted structural power metrics and operated under uniform grid profiles.

Policy-gradient methods have also been explored for scheduling tasks under fluctuating loads. Proximal Policy Optimisation (PPO) [4] provides stable on-policy updates via a clipped surrogate objective and has shown strong convergence in diverse discrete allocation tasks. Prior comparisons between value-based and policy-gradient agents in multi-objective scheduling contexts remain sparse, motivating our direct DQN vs. PPO benchmark. Crucially, as Section **??** demonstrates, this benchmark yields a counterintuitive result where the simpler off-policy value method converges more rapidly under strict sample and time limits than its modern policy-gradient counterpart.

Energy-aware scheduling has concurrently emerged as a critical research direction as data-centre power consumption becomes a dominant operational and environmental cost. Multi-objective formulations that jointly minimize processing latency and infrastructure power footprint have been studied using classical evolutionary algorithms and, more recently, deep reinforcement learning variants tracking dynamic CPU utilization profiles. Our work directly bridges these two disconnected streams, extending the core scheduling logic of the DDQN-TS lineage into an energy-efficient paradigm by introducing a multi-objective, power-aware Gymnasium simulator.

A. Reward Functions

The baseline reward (Tong et al.) is a bi-objective scalar penalising response time and resource imbalance. Our proposed reward upgrades this to a multi-objective, energy-aware framework that jointly optimises task throughput T (tasks/sec), processing latency L (ms), and infrastructure energy cost E (Watts). To resolve the dimensional mismatch and scaling inconsistencies across these heterogeneous metrics, we enforce objective-level Min-Max normalization. Crucially, rather than utilizing dynamic running statistics which introduce destabilizing non-stationarity into the reinforcement signal, we compute fixed, analytically-derived theoretical bounds ($T_{\min}, T_{\max}, L_{\min}, L_{\max}, E_{\min}, E_{\max}$) directly from the environment's structural and physical infrastructure constraints. The normalized metrics are explicitly bounded via a clipping operation to $[0, 1]$ before weight application, ensuring absolute gradient stability during volatile workload spikes. The operational scalarized reinforcement signal is mathematically formulated as:

$$R_t = c \cdot (w_1 \cdot T^* - w_2 \cdot L^* - w_3 \cdot E^*)$$

where $T^* = \text{clip}\left(\frac{T - T_{\min}}{T_{\max} - T_{\min}}, 0, 1\right)$, and analogously for L^* and E^*, under the convex weight constraint $w_1 + w_2 + w_3 = 1$. Furthermore, we introduce an empirical reward scaling factor $c = 10.0$. This scaling multiplier does not alter the underlying multi-objective Pareto landscape, as it represents a strict monotonic transformation that preserves policy ranking over the weight simplex, but it successfully amplifies initial sparse gradient signals. This modification significantly accelerates reinforcement learning convergence and prevents premature policy stagnation during early exploration phases. The default operational weight configuration is set to $w_1 = 0.50$, $w_2 = 0.30$, and $w_3 = 0.20$, representing a balanced paradigm that prioritizes high throughput while systematically penalizing latency degradation and excessive power allocation.

III. Methodology

A. Phase 0 — Baseline Replication (DDQN-TS)

Objective. Phase 0 reproduces the baseline algorithm from Tong et al. [1] to establish a verified performance floor before introducing our proposed modifications. To comply with modern library standards and ensure strict comparative validity, the original hand-rolled TensorFlow/Keras implementation was ported onto a unified `gymnasium.Env` interface, with agents trained via `stable_baselines3` (SB3). All scheduling mathematics — including the task/VM configuration, the strict SLA gate constraints, execution/wait/response-time definitions, the bi-objective reward formulation, and the final TCR/ART evaluation metrics — were mapped term-by-term from Eqs. (1)–(16), Algorithm 1, and Table 3 of the source paper [1] without reinterpretation.

a) Environment.: The baseline environment (`CloudSchedulingEnv`) implements an online decision loop where one training episode represents a single deterministic pass through a fixed task sequence, strictly replicating the sequential flow of State $\rightarrow$ Action $\rightarrow$ SLA Gate Verification $\rightarrow$ Reward Evaluation $\rightarrow$ Infrastructure State Update:

- **State:** A localized vector tracking per-VM available capacities and cumulative idle times, $S_t = [\text{VM\_idle}_1, \ldots, \text{VM\_idle}_m]$.
- **Action:** Discrete(m), designating which specific Virtual Machine is allocated the incoming task.

- **Reward:** The joint SLA-gated, bi-objective optimization reward defined in Eq. (16) of [1].
- **SLA Gate:** A mathematical constraint where a task is accepted by a VM if and only if:

$$\text{VM\_idle} + T_{\text{exe}} \leq T_{\text{arrival}} + \frac{T_{\text{size}}}{\text{VM\_speed}_{\text{slow}}}$$

(per Eqs. 7–8 in [1]).

For validation, the architecture models the paper's primary 4-VM topology (configured at processing speeds of 200/800/1500/2000 MIPS).

b) Workloads.: Three statistically-matched task generators were constructed to represent diverse operational profiles: a synthetic *Random* workload (uniform size distribution, static arrival windows), a heavy-tailed *Google-proxy* workload (task sizes sampled from a Pareto$(\text{shape} = 1.5)$ distribution to mimic cloud job behaviors as characterized in empirical cluster studies [2]), and a bimodal *Alibaba-proxy* mixture (an 80/20 split between standard short jobs $U[50, 500]$ and long-burst compute loads $U[1000, 3000]$). These generators act as statistically rigorous proxies rather than ingesting raw industrial trace archives directly.

c) Agents and Traditional Baselines.: Two learned value-based policies are evaluated against non-learned baselines (Random allocation and Round-Robin). Both neural agents implement a multi-layer perceptron (2 hidden layers $\times$ 50 ReLU units), an SGD optimizer (lr $= 0.1$), discount factor $\gamma = 0.9$, batch size 32, a replay capacity of 10,000, and a constant exploration rate $\varepsilon = 0.5$. Due to SB3's native configuration constraints regarding non-annealed schedules, this was enforced by setting the initial and final exploration parameters symmetrically:

- **DQN:** Stock Stable-Baselines3 Deep Q-Network, utilizing a target network refresh interval of 1,000 steps to enforce default baseline policy stability.
- **DDQN-TS:** A customized `GenuineDDQN` subclass overriding SB3's underlying target value calculation. Because stock SB3 computes standard bootstrap targets ($y = r + \gamma \cdot \max_a Q_{\text{target}}(s', a)$), `GenuineDDQN` strictly overrides the `train()` target estimation loop to implement true Double DQN logic per Eq. (15) of [1] — isolating action selection via the online network from action evaluation via the target network — while preserving SB3's replay data loops. To adhere to Table 3 of the source text, its target refresh frequency is set to $N = 20$. This deliberate asymmetry in update intervals ($1,000$ vs. 20) is maintained to preserve the exact operational parameters established by Tong et al. [1] while preventing standard baseline divergence under severe scaling.

d) Training and Evaluation Protocol.: Agents are trained across a paper-scale episode of 50,000 continuous tasks. Performance validation is executed under fully deterministic, greedy deployment settings against unseen validation blocks spanning sizes $n \in \{1000, 2000, 4000, 8000\}$. The primary scientific metrics recorded are Task Completion Rate (TCR) and Average Response Time (ART).

e) Disclosed Limitations.: We explicitly note three execution divergences from the source literature:

(1) SB3 natively binds optimization loss to Huber/Smooth-L1 loss instead of standard Mean Squared Error (MSE).
(2) Exploration follows a constant $\varepsilon = 0.5$ boundary condition since the source paper does not specify an explicit temporal annealing curve.
(3) Training occurs on a fixed-sequence task arrival array, causing agents to bind behavioral parameters closely to specific sequential distributions.

B. Phase 1 — Proposed Framework and Multi-Scenario Extensions

Objective. Phase 1 shifts from the baseline replication toward our enhanced energy-efficient cloud scheduling framework. Grounded in multi-objective resource optimization theory [3], it introduces a richer, high-fidelity cluster model that optimizes a tri-objective space (Throughput, Latency, and Energy-efficiency) under dynamically fluctuating load patterns. To thoroughly evaluate the framework's robustness, Phase 1 encompasses the primary multi-objective environment alongside a suite of experimental stress tests, including large-scale network extensions, heavy-tailed blocking scenarios, asset energy asymmetries, and transient system resilience.

a) Environment.: The primary Phase 1 environment instantiates a dynamic cluster structure characterized by:

- **Cluster Model:** An infrastructure suite consisting of $N = 4$ heterogeneous physical servers. Each server $i \in \{1, \ldots, N\}$ possesses independent CPU capacity C_i^{cpu} and RAM capacity C_i^{ram} (normalized to 100 units). Following standard industrial hardware power models [4], each node draws an idle base power of $P_{\text{idle}} = 100$W and scales linearly up to a peak power boundary of $P_{\text{max}} = 250$W under full processing load, such that instantaneous power consumption scales with CPU utilization:

$$P_i(t) = P_{\text{idle}} + (P_{\text{max}} - P_{\text{idle}}) \cdot U_i^{\text{cpu}}(t)$$

- **Task Model:** Tasks arrive via an un-batched Poisson process with a configurable background arrival rate λ, a standard formulation for capturing stochastic network queuing behaviors [5]. Incoming tasks demand variable capacities ($U[5, 30]\%$ of CPU/RAM), specify computational execution scales ($U[1, 10]$ simulation steps), and hold discrete priority metrics $\kappa \in \{1, 2, 3\}$. Each server maintains a bounded processing backlog pipeline (maximum queue capacity $Q_{\text{max}} = 10$).
- **State Space:** A 26-dimensional observation vector ($5 \cdot N + 6$ where $N = 4$ servers), programmatically mapping individual server CPU metrics, RAM metrics, current power profiles, normalized queue backlogs, and total outstanding work weights, augmented by 6 overarching global parameters detailing incoming job metrics and aggregate system load contexts.
- **Action Space:** Discrete(N), mapping the structural routing decision to allocate the current task to a targeted server

queue. If the chosen queue is full ($Q_i(t) = Q_{\max}$), the task is dropped, incurring an explicit throughput penalty.

- **Reward Structure:** A commensurable, normalized tri-objective reward function:

$$R = \text{reward\_scale} \cdot (w_1 \cdot \text{throughput\_norm} - w_2 \cdot \text{latency\_norm} - w_3 \cdot \text{energy\_norm})$$

where reward_scale $= 10.0$, and default structural weights are balanced at $(0.5, 0.3, 0.2)$. Each internal objective is dynamically normalized via min-max boundaries derived from analytical system limits.

b) Core Agents and Algorithmic Baselines.: Advanced value-based (DQN [6]) and policy-gradient (PPO [7]) agents are trained within the primary cluster environment for 200,000 steps at a baseline arrival intensity ($\lambda = 1.0$). These are evaluated against four operational heuristics, formalized as follows:

- **Round-Robin:** Allocates incoming task t to server $i = (t \bmod N) + 1$, completely independent of infrastructure state metrics.
- **Random Routing:** Uniformly samples an action $i \sim U\{1, N\}$.
- **Queue-Load Shortest-Job-First (SJF):** Selects the target server i^* that minimizes a localized performance index balancing current backlog length and task properties, minimizing $\arg\min_i [Q_i(t) \cdot T_{\text{exe}}]$.
- **Greedy Min-Min:** Computes the expected job completion time across all available nodes and greedily routes the incoming task to the node yielding the earliest absolute availability time:

$$i^* = \arg\min_i [W_i(t) + T_{\text{exe},i}]$$

where $W_i(t)$ represents the cumulative outstanding work time remaining in queue i.

c) Experimental Variations and Multi-Scenario Extensions.: To comprehensively test policy limits beyond standard baseline conditions, Phase 1 includes four localized environmental configurations:

- **Load Stress Testing:** Evaluation of all core policies across six background Poisson arrival rates, $\lambda \in \{0.0, 0.5, 1.0, 1.5, 2.0, 3.0\}$, mapping structural behaviors from light demand up to severe cluster saturation.
- **Reward Weight Sensitivity Analysis:** To evaluate the tri-objective trade-off space, pre-trained models undergo 30,000 steps of fine-tuning under four specialized reward alignments: Throughput-Priority $(0.7/0.2/0.1)$, Latency-Priority $(0.2/0.6/0.2)$, Energy-Priority $(0.2/0.2/0.6)$, and Equal Objective Weighting $(0.33/0.33/0.34)$, validated across 10 evaluation trials.
- **Scale and Topology Resilience (8-Server Extension):** Scaling the core environment setup to $N = 8$ physical servers with heterogeneous processing and capacity bounds. This experiment expands the observation space to 46 dimensions and the action space to Discrete(8), testing the structural generalization of the deep RL models.
- **Transient Failure and Power Asymmetries:** Focused boundary experiments that introduce mid-episode server crashes to measure structural fault recovery, and highly asymmetric power profiles to verify if the reward function prevents agent utilization of inefficient hardware nodes during standard demand periods.

d) Disclosed Limitations.: To preserve full scientific transparency, three critical limitations of the Phase 1 setup are noted:

(1) **Single Training Seed Constraints:** All core models (DQN, PPO) are trained using a single static random seed (seed $= 42$). While evaluation runs span multiple statistical validation episodes, the underlying structural policy variance across multiple decoupled training trajectories remains unquantified, masking potential training initialization dependencies common in deep RL architectures.

(2) **Homogeneous Queue Limits:** The environment enforces symmetric queue capacities ($Q_{\max} = 10$) across all infrastructure nodes, simplifying realistic cloud configurations where larger physical nodes maintain proportionally larger buffers.

(3) **Linear Power Approximations:** Node power draw scales via strict linear interpolation; non-linear dynamic voltage and frequency scaling (DVFS) curve anomalies are not simulated.

IV. Results

A. Baseline Replication Results

The baseline implementation reproduced the DDQN-TS cloud scheduling framework using Stable-Baselines3 and evaluated its performance against the original Deep Q-Network (DQN) and traditional heuristic scheduling algorithms. The experiments were conducted using both the original 4-VM topology and an extended 8-VM topology, while maintaining the workload generation and evaluation methodology described in the reference paper.

a) Training Convergence.: The convergence curves demonstrate that both reinforcement learning agents successfully learned effective scheduling policies during training. However, the proposed Genuine DDQN-TS exhibited a more stable learning process than the standard DQN.

For the 4-VM environment, DDQN-TS achieved smoother reward convergence with lower oscillation throughout training, whereas the vanilla DQN showed larger fluctuations before approaching convergence. This behavior indicates that the Double DQN update mechanism reduces the overestimation bias commonly observed in standard DQN, leading to more stable policy learning.

When the infrastructure was expanded to 8 virtual machines, the increased scheduling complexity naturally slowed convergence for both agents. Nevertheless, DDQN-TS maintained stable learning behavior and adapted effectively to the larger action space, while vanilla DQN experienced greater reward instability during training.

B. Task Completion Rate (TCR)

Task Completion Rate (TCR) was evaluated under three workload distributions (Random, Google-like, and Alibaba-like) across increasing workload sizes.

The results show that both reinforcement learning approaches consistently outperformed the heuristic scheduling algorithms, including Round Robin and Random scheduling. Among the learning-based methods, DDQN-TS achieved the highest task completion rates across all workload scenarios.

As the number of submitted tasks increased, the performance gap between DDQN-TS and the heuristic schedulers became more pronounced, demonstrating the ability of reinforcement learning to make more effective scheduling decisions under heavy workloads. The Google and Alibaba benchmark traces produced similar trends, indicating that the learned policy generalizes well across different workload characteristics.

C. Average Response Time (ART)

Average Response Time was measured using the same benchmark workloads to evaluate scheduling latency.

The reinforcement learning schedulers consistently achieved lower response times than the heuristic baselines. DDQN-TS produced the lowest average response time across nearly all workload sizes and benchmark datasets.

Although response time increased as workload intensity grew, the increase was significantly smaller for DDQN-TS than for the traditional scheduling methods. This suggests that the learned scheduling policy allocates incoming tasks more efficiently, reducing waiting time and improving overall system responsiveness.

D. Scalability Evaluation

The scalability experiment extended the cloud environment from 4 virtual machines to 8 virtual machines while maintaining identical workload conditions.

The expanded infrastructure increased overall scheduling capacity and reduced queue congestion. Both reinforcement learning agents benefited from the additional computational resources; however, DDQN-TS continued to outperform vanilla DQN by achieving higher task completion rates and lower response times.

The results demonstrate that the learned policy scales effectively with increasing resource availability while maintaining stable performance under larger scheduling action spaces.

E. Overall Baseline Performance

The baseline replication successfully reproduced the primary findings reported in the original DDQN-TS study. Across all benchmark workloads, DDQN-TS consistently achieved:

- More stable convergence during training than vanilla DQN.
- Higher Task Completion Rate (TCR) than both DQN and heuristic scheduling algorithms.
- Lower Average Response Time (ART) across all workload distributions.
- Strong scalability when the infrastructure expanded from 4 to 8 virtual machines.

These baseline results establish a reliable reference for evaluating the proposed modifications introduced in the subsequent experimental phases. The reproduced framework confirms that the Stable-Baselines3 implementation preserves the performance characteristics of the original DDQN-TS algorithm and provides a solid foundation for further experimentation.

TABLE I
OVERALL BASELINE PERFORMANCE (AVERAGE ACROSS ALL WORKLOADS)

Algorithm	Average Task Completion Rate (TCR)	Average Response Time (ART) (s)
DDQN-TS	0.765	2.067
DQN	0.822	1.597
Random	0.843	1.038
Round Robin	0.881	0.981

F. Phase 1: Baseline Deep Reinforcement Learning Scheduler

The baseline experiment evaluated the proposed cloud scheduler using DQN and PPO agents trained for 200,000 timesteps. Their performance was compared against four traditional scheduling algorithms (Round Robin, Shortest Job First, Greedy Min-Min, and Random) under workload arrival rates ranging from $\lambda = 0.0$ to $\lambda = 3.0$ tasks per time unit.

The results demonstrate that the reinforcement learning agents successfully learned scheduling policies capable of balancing multiple objectives, including throughput, latency, and energy consumption. As workload intensity increased, both DQN and PPO maintained higher cumulative rewards than the heuristic approaches while keeping energy consumption relatively stable.

The reward-weight sensitivity analysis further showed that the learned policies could adapt to different optimization priorities. Configurations emphasizing throughput achieved the highest average rewards, while balanced configurations provided the best trade-off between latency and energy efficiency. Overall, the baseline confirms that deep reinforcement learning can outperform conventional scheduling heuristics in multi-objective cloud scheduling.

TABLE II
OVERALL PERFORMANCE EVALUATION ACROSS DIFFERENT SCHEDULING POLICIES

Scheduling Policy	Mean Reward	Mean Throughput	Mean Latency (ms)	Mean Energy (W)
DQN	147.77	2.13	6564.38	658.23
PPO (Proposed)	143.64	2.15	6828.69	655.53
Round Robin	159.47	2.07	4891.43	663.12
Random	158.68	2.06	4994.17	662.82
Greedy Min-Min	162.23	2.14	4496.16	664.50
SJF	162.23	2.14	4496.16	664.50

G. Experiment 1: Heavy Workload Stress Test

To evaluate robustness under extreme operating conditions, the workload distribution was modified to generate significantly longer tasks. This experiment tested whether the learned scheduling policy could maintain stable performance under heavy computational demand.

The results indicate that the reinforcement learning agents continued to outperform the heuristic schedulers despite the increased workload complexity. Although latency naturally increased because of longer execution times, the RL policies maintained higher throughput and better overall rewards by distributing tasks more efficiently across available virtual machines. This demonstrates that the proposed scheduler generalizes well beyond the workload characteristics observed during normal training.

H. Experiment 2: Increased Server Scale (Eight-Server Environment)

The second experiment expanded the cloud infrastructure from the baseline configuration to an environment containing eight servers ($N = 8$). This evaluated whether the scheduler could scale with increasing resource availability and a larger action space.

The experimental results show that both DQN and PPO adapted successfully to the larger infrastructure. The scheduler effectively utilized the additional computing resources, reducing queue buildup while maintaining stable energy consumption. Despite the increased complexity of the scheduling problem, the reinforcement learning agents preserved their performance advantage over the heuristic baselines, demonstrating good scalability.

TABLE III
PERFORMANCE EVALUATION ACROSS SCHEDULING POLICIES UNDER ENERGY ASYMMETRY SCENARIO

Scheduling Policy	Mean Reward	Mean Throughput	Mean Latency (ms)	Mean Energy (W)
DQN	79.42	2.15	8472.80	1054.39
PPO (Proposed)	113.89	2.27	3794.08	1071.31
Round Robin	115.02	2.28	3634.70	1071.78
Random	114.94	2.27	3650.03	1071.74
Greedy Min-Min	115.24	2.28	3612.33	1071.91
SJF	115.24	2.28	3612.33	1071.91

I. Experiment 3: Heterogeneous Energy Consumption

Unlike the baseline where servers shared identical power characteristics, this experiment introduced heterogeneous virtual machines with significantly different static and dynamic power consumption profiles.

The reinforcement learning agents learned to exploit these differences by preferentially assigning workloads to more energy-efficient servers whenever possible while reserving high-power servers for demanding tasks. Consequently, the scheduler achieved a better energy-latency trade-off than the heuristic methods.

The reward sensitivity analysis also showed that different objective weight configurations shifted the scheduling behavior as expected. Throughput-oriented configurations achieved the highest rewards, whereas balanced and latency-oriented settings produced more energy-efficient operating points without substantial performance degradation. These findings demonstrate that the proposed approach can effectively optimize scheduling decisions in heterogeneous cloud environments.

TABLE IV
OVERALL PERFORMANCE EVALUATION ACROSS DIFFERENT SCHEDULING POLICIES (PHASE 1 BASELINE)

Scheduling Policy	Mean Reward	Mean Throughput	Mean Latency (ms)	Mean Energy (W)
DQN	147.28	2.17	6530.31	658.24
PPO (Proposed)	122.74	2.09	9478.86	647.87
Round Robin	159.47	2.21	4891.43	663.12
Random	158.68	2.21	4994.17	662.82
Greedy Min-Min	162.23	2.22	4496.16	664.50
SJF	162.23	2.22	4496.16	664.50

J. Experiment 4: Transient Server Failure

To evaluate fault tolerance, a temporary server failure was injected during execution by making one virtual machine unavailable for a fixed time interval.

The reinforcement learning scheduler rapidly adapted to the loss of computing resources by redirecting incoming tasks toward the remaining operational servers. Queue lengths temporarily increased immediately after the failure but recovered once the scheduling policy redistributed the workload. The system maintained continuous task processing throughout the failure period, indicating that the learned policy possesses resilience against transient infrastructure faults.

TABLE V
PERFORMANCE AND FAULT TOLERANCE EVALUATION ACROSS SCHEDULING POLICIES UNDER TRANSIENT FAILURE (RESILIENCE SCENARIO)

Scheduling Policy	Mean Reward	Mean Throughput	Mean Latency (ms)	Mean Energy (W)	Mean Fault Violations
DQN	54.12	2.06	8579.42	683.02	8.41
PPO (Proposed)	-16.91	2.01	12637.39	674.22	18.18
Round Robin	15.82	2.00	10687.63	689.46	15.00
Random	7.91	2.00	11152.74	688.70	15.95
Greedy Min-Min	105.19	2.08	6386.28	691.73	0.69
SJF	105.19	2.08	6386.28	691.73	0.69

K. Experiment 5: Multi-Objective Reward Sensitivity Analysis

The final experiment investigated the influence of different reward weight configurations on scheduling behavior. Multi-

ple optimization profiles — including balanced, throughput-priority, latency-priority, and energy-priority — were evaluated.

The results confirm that the reward function successfully guides policy behavior toward the selected objective. Throughput-priority configurations consistently produced the highest cumulative rewards and task completion rates. Energy-priority configurations reduced overall power consumption at the cost of slightly increased latency, while balanced configurations achieved the most favorable compromise between throughput, latency, and energy consumption.

These observations verify that the proposed reinforcement learning framework is highly configurable and can be adapted to different cloud management objectives without requiring changes to the underlying scheduling algorithm.

V. GenAI Usage Disclosure

In accordance with the GenAI use policy, I acknowledge the use of generative AI tools (such as Gemini) primarily for concept clarification and code optimization. Specifically, the AI was utilized to refine the existing code for better readability and to apply clean code principles, ensuring the implementation is efficient and maintainable. All core logic and experimental setups were developed independently, and the AI's role was limited to grammar enhancements and minor stylistic edits to the report, ensuring the authentic representation of the research findings and my personal understanding of the project.

References

[1] Z. Tong, F. Ye, B. Liu, et al., "DDQN-TS: A novel bi-objective intelli gent scheduling algorithm in the cloud environment," Neurocomputing, vol. 455, pp. 419–430, 2021.

[2] M. Masdari, S. S. Nabavi, and V. Ahmadi, "An overview of virtual machine placement schemes in cloud computing," J. Netw. Comput. Appl., vol. 66, pp. 106–127, 2016.

[3] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in Proc. 15th ACM Workshop Hot Topics Netw. (HotNets), Atlanta, GA, USA, 2016

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint, arXiv:1707.06347, 2017.